Day-32: Docker volumes and Networking

Task1: Task 1: The Problem Run a Postgres or MySQL container Create some data inside it (a table, a few rows — anything) Stop and remove the container Run a new one — is your data still there? Write what happened and why.

1.Step 1 — Run PostgreSQL container

docker run -d \

--name postgres-test \

-e POSTGRES_PASSWORD=pass123 \

-p 5432:5432 \

postgres , check container : docker ps

2. Step 2 — Enter the container : docker exec -it postgres-test psql -U postgres

Step 3 — Create table + insert data :

CREATE TABLE students (

id SERIAL PRIMARY KEY,

name TEXT

);

INSERT INTO students (name) VALUES ('Padma');

INSERT INTO students (name) VALUES ('DevOps');

SELECT * FROM students;

Expected output:

```
id |  name
----+--------
 1  | Padma
 2  | DevOps
```

Step 4: Stop container and remove container : docker stop id && docker rm id

Step 6 — Run NEW container again

Step 7 : Check if data exists Enter container again:

`docker exec -it postgres-test psql -U postgres`

Run:`SELECT * FROM students;`

You will see: `ERROR: relation "students" does not exist`

Postgres stored data inside the container filesystem:/var/lib/postgresql/data

When the container was deleted:container filesystem = deleted
data = deleted ,This is why **containers are ephemeral**.This problem is exactly why **Docker volumes exist**.

```
ubuntu@ip-172-31-44-85: ~
Digest: sha256:69e8582b781cb44fa4557b98ed586fe68361e320d9b12f9707494335634f4f3d
Status: Downloaded newer image for postgres:latest
a22a5a04e8bd8f35d97bfe9d2145719a659fdd35c6628beff7ad05dd4bc0f3e6
ubuntu@ip-172-31-44-85:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED         STATUS         PORTS
                         NAMES
a22a5a04e8bd   postgres   "docker-entrypoint.s…"   3 seconds ago   Up 2 seconds   0.0.0.0:5432->5432/t
p, [::]:5432->5432/tcp   postgres-test
ubuntu@ip-172-31-44-85:~$ docker exec -it postgres-test psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

postgres=# CREATE TABLE students (
id SERIAL PRIMARY KEY,
name TEXT
);

INSERT INTO students (name) VALUES ('Padma');
INSERT INTO students (name) VALUES ('DevOps');

SELECT * FROM students;
CREATE TABLE
INSERT 0 1
INSERT 0 1
 id |  name
----+--------
  1 | Padma
  2 | DevOps
(2 rows)

postgres=# /q
postgres-# exit
Use \q to quit.
postgres-# \q
ubuntu@ip-172-31-44-85:~$ docker stop postgres-test
postgres-test
ubuntu@ip-172-31-44-85:~$ docker rm postgres-test
postgres-test
ubuntu@ip-172-31-44-85:~$ docker run -d \
--name postgres-test \
-e POSTGRES_PASSWORD=pass123 \
-p 5432:5432 \
postgres
b4789201743271949fc8b5252dc01166ec09a62386bb1d63ca93be8a8a9e7cd9
ubuntu@ip-172-31-44-85:~$ docker exec -it postgres-test psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

postgres=# SELECT * FROM students;
ERROR:  relation "students" does not exist
LINE 1: SELECT * FROM students;
                      ^
postgres=# /q
postgres-# exit
Use \q to quit.
postgres-# \q
```

Task 2: Named Volumes Create a named volume Run the same database container, but this time attach the volume to it Add some data, stop and remove the container Run a brand new container with the same volume Is the data still there? Verify: docker volume ls, docker volume inspect.

1.create a volume : docker volume create postgres-data and verify it : docker volumes ls

2.Run Container with mount path :

docker run -d \

--name postgres-vol \

-e POSTGRES_PASSWORD=pass123 \

-v postgres-data:/var/lib/postgresql \

-p 5432:5432 \

Postgres

3.verify the container is running : docker ps

4. Connect to database : docker exec -it postgres-vol psql -U postgres you can see Prompt:

postgres=#

5.create data again : take same we used in task 1

6.stop container and delete : docker stop postgres-vol && docker rm postgres-vol

7. Start new container using same volume: take the same container used in step 2.

8. check data : docker exec -it postgres-vol2 psql -U postgres and Run : SELECT * FROM devops;

You can still see the Padma and Devops.

That proves **volume persistence works**. Data remained after container deletion.

Reason : Docker volume stores data **outside container filesystem**.

Container deleted → Data safe.

Volume persists → Data persists.

```
ubuntu@ip-172-31-44-85: ~
a3b88592774da862edd63448764a5566bcd1c13b0f658671511c27eb028fe8fe
ubuntu@ip-172-31-44-85:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED         STATUS         PORTS
                          NAMES
a3b88592774d   postgres   "docker-entrypoint.s…"   4 seconds ago   Up 4 seconds   0.0.0.0:5432->54
p, [::]:5432->5432/tcp   postgres-vol
ubuntu@ip-172-31-44-85:~$ docker exec -it postgres-vol psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

postgres=# CREATE TABLE devops (
id SERIAL PRIMARY KEY,
tool TEXT
);

INSERT INTO devops (tool) VALUES ('Docker');
INSERT INTO devops (tool) VALUES ('Kubernetes');

SELECT * FROM devops;
CREATE TABLE
INSERT 0 1
INSERT 0 1
 id |    tool
----+------------
  1 | Docker
  2 | Kubernetes
(2 rows)

postgres=# \q
ubuntu@ip-172-31-44-85:~$ docker stop postgres-vol
postgres-vol
ubuntu@ip-172-31-44-85:~$ docker rm postgres-vol
postgres-vol
ubuntu@ip-172-31-44-85:~$ docker run -d \
--name postgres-vol2 \
-e POSTGRES_PASSWORD=pass123 \
-v postgres-data:/var/lib/postgresql \
-p 5432:5432 \
postgres
4d23f910fe33f6fbb7d8a8b1481a2ae37b1599eed73db94b922e47b8b0804626
ubuntu@ip-172-31-44-85:~$ docker exec -it postgres-vol2 psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

postgres=# docker exec -it postgres-vol2 psql -U postgres
postgres-# SELECT * FROM devops;
ERROR:  syntax error at or near "docker"
LINE 1: docker exec -it postgres-vol2 psql -U postgres
        ^
postgres=# SELECT * FROM devops;
 id |    tool
----+------------
  1 | Docker
  2 | Kubernetes
(2 rows)
```

Task 3: Bind Mounts Create a folder on your host machine with an index.html file Run an Nginx container and bind mount your folder to the Nginx web directory Access the page in your browser Edit the index.html on your host — refresh the browser Write in your notes: What is the difference between a named volume and a bind mount?

1.create a folder : mkdir mywebsite, cd mywebsite, vim index.html: <h1>Hello from Docker Bind Mount</h1>

2. Run Nginx container with bind mount

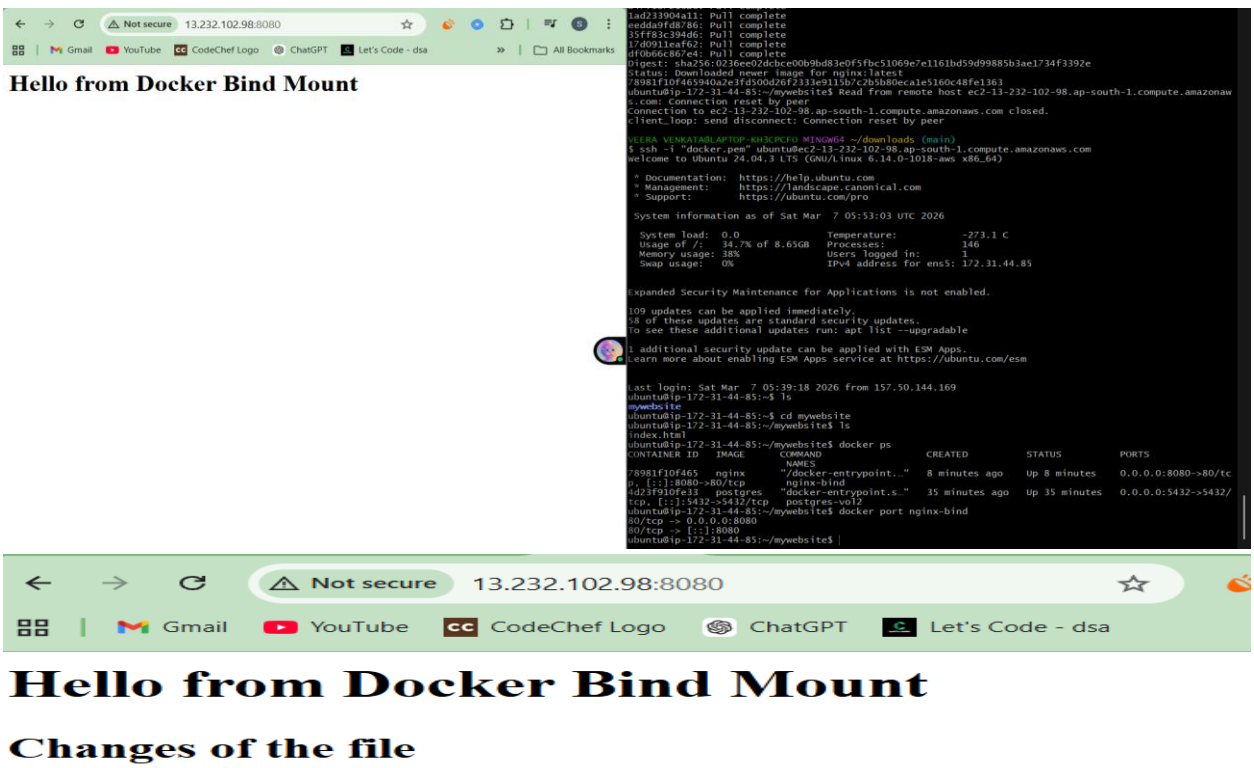
```
docker run -d \
--name nginx-bind \
-p 8080:80 \
-v $(pwd):/usr/share/nginx/html \
nginx
```

3.Access in browser: http://EC2_PUBLIC_IP:8080

Changes made on the host machine were **immediately reflected in the container.**

Explanation :

Bind mounts directly connect a **host directory to a container directory,** allowing real-time file updates.



Task 4: Docker Networking Basics List all Docker networks on your machine Inspect the default bridge network Run two containers on the default bridge — can they ping each other by name? Run two containers on the default bridge — can they ping each other by IP?

1. List all Docker networks on your machine :docker network ls

2. Inspect the default bridge network : docker network inspect bridge

You will see a large JSON output, Important section looks like:

```
"Subnet": "172.17.0.0/16"
"Gateway": "172.17.0.1"
```

This means Docker automatically creates a **private network**.

3. Run two containers on the default bridge — can they ping each other by name: NO

4. Run two containers on the default bridge — can they ping each other by IP: YES

The default bridge network **does not provide automatic DNS-based name resolution**.

Containers can communicate **only using IP addresses** unless a **custom network** is used.

```
ubuntu@ip-172-31-44-85:~$ docker inspect container1 | grep IPAddress
            "SecondaryIPAddresses": null,
            "IPAddress": "172.17.0.4",
                    "IPAddress": "172.17.0.4",
ubuntu@ip-172-31-44-85:~$ docker inspect container2 | grep IPAddress
            "SecondaryIPAddresses": null,
            "IPAddress": "172.17.0.5",
                    "IPAddress": "172.17.0.5",
ubuntu@ip-172-31-44-85:~$ docker exec -it container1 bash
root@bb8ef65c8ebd:/# ping 172.17.0.4
PING 172.17.0.4 (172.17.0.4) 56(84) bytes of data.
64 bytes from 172.17.0.4: icmp_seq=1 ttl=64 time=0.030 ms
64 bytes from 172.17.0.4: icmp_seq=2 ttl=64 time=0.029 ms
64 bytes from 172.17.0.4: icmp_seq=3 ttl=64 time=0.032 ms
64 bytes from 172.17.0.4: icmp_seq=4 ttl=64 time=0.049 ms
64 bytes from 172.17.0.4: icmp_seq=5 ttl=64 time=0.030 ms
64 bytes from 172.17.0.4: icmp_seq=6 ttl=64 time=0.033 ms
^C
--- 172.17.0.4 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5121ms
rtt min/avg/max/mdev = 0.029/0.033/0.049/0.006 ms
root@bb8ef65c8ebd:/#
```

```
ubuntu@ip-172-31-44-85: ~
ubuntu@ip-172-31-44-85:~$ docker run -it --name c2 busybox sh
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
61dfb50712f5: Pull complete
Digest: sha256:b3255e7dfbcd10cb367af0d409747d511aeb66dfac98cf30e97e87e4207dd76f
Status: Downloaded newer image for busybox:latest
/ # ping c1
ping: bad address 'c1'
/ # exit
ubuntu@ip-172-31-44-85:~$ docker run -it --nmae busybox sh
unknown flag: --nmae

Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Run 'docker run --help' for more information
ubuntu@ip-172-31-44-85:~$ docker run -it --name busybox sh
Unable to find image 'sh:latest' locally
docker: Error response from daemon: pull access denied for sh, repository does not exist or may require
 'docker login': denied: requested access to the resource is denied

Run 'docker run --help' for more information
ubuntu@ip-172-31-44-85:~$ docker run -it --name c1  busybox sh
/ # ping c2
ping: bad address 'c2'
/ # exit
ubuntu@ip-172-31-44-85:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
d3a5fdf24151   bridge    bridge    local
a363265cd5b7   host      host      local
492d21537fd7   none      null      local
ubuntu@ip-172-31-44-85:~$ docker network inspect bridge
[
    {
        "Name": "bridge",
        "Id": "d3a5fdf24151c6231291aacef490b1715f57bda64b73cd45ffd70d5c3591bde2",
        "Created": "2026-03-07T04:55:49.606031446Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.17.0.0/16",
                    "Gateway": "172.17.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
```

Task 5: Custom Networks Create a custom bridge network called my-app-net Run two containers on my-app-net Can they ping each other by name now? Write in your notes: Why does custom networking allow name-based communication but the default bridge doesn't?

1. Custom Networks Create a custom bridge network called my-app-net: docker network create my-app-net and verify it : docker network ls

2. Run first container on this network : docker run -dit \

--name app1 \

--network my-app-net \

Ubuntu

3. Run second container on same network : docker run -dit \

--name app2 \

--network my-app-net \

Ubuntu

4. Check containers: `docker ps`

5. Install ping tool: Enter container **app1** :docker exec -it app1 bash

Install ping:apt update , apt install iputils-ping -y

6. Ping using container name: ping app2

Docker custom networks include an **internal DNS server** that automatically resolves container names to their IP addresses.

```
root@172d3b05f66f:/# ping app2
PING app2 (172.18.0.3) 56(84) bytes of data.
64 bytes from app2.my-app-net (172.18.0.3): icmp_seq=1 ttl=64 time=0.065 ms
64 bytes from app2.my-app-net (172.18.0.3): icmp_seq=2 ttl=64 time=0.050 ms
64 bytes from app2.my-app-net (172.18.0.3): icmp_seq=3 ttl=64 time=0.051 ms
^C
--- app2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2041ms
rtt min/avg/max/mdev = 0.050/0.055/0.065/0.006 ms
root@172d3b05f66f:/#
```

Why does custom networking allow name-based communication but the default bridge doesn't?

-> Custom Docker bridge networks provide built-in DNS-based service discovery, allowing containers to communicate using container names. The default bridge network does not support automatic DNS name resolution between containers.

Task 6: Task 6: Put It Together Create a custom network Run a database container (MySQL/Postgres) on that network with a volume for data Run an app container (use any image) on the same network Verify the app container can reach the database by container name.

1.Create network: docker network create my-app-net

2. Create a volume for database: docker volume create mysql-data

3.Run MySQL container: bash

```
docker run -d \
--name mysql-db \
--network my-app-net \
-e MYSQL_ROOT_PASSWORD=root123 \
-v mysql-data:/var/lib/mysql \
mysql
```

4. Run application container : bash

```
docker run -dit \
--name app-container \
--network my-app-net \
ubuntu
```

5.Install mysql client inside app container: - Enter container:docker exec -it app-container bash - Install mysql client:apt update ,apt install mysql-client -y

6. Connect to database using container name - Inside container run:mysql -h mysql-db -u root -pPassword:root123

7.If connection succeeds you will see: mysql>